

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина: Бэк-энд разработка**

Отчет

Домашнее задание 6

Автоматический деплой backend-приложения

Выполнил:

Зеленков Евгений

БР2.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

## ДЗ6: Автоматический деплой backend-приложения

### Цель работы

Настроить автоматическое развёртывание приложения из ЛР4 на удалённый сервер при обновлении кода в отдельной ветке репозитория.

### Итоговая схема

Для ДЗ6 создана отдельная ветка hw6. Приложение и все необходимые для деплоя файлы вынесены в отдельную директорию:

```
BP2.2/Зеленков Евгений/homeworks/hw6/app
```

Workflow GitHub Actions находится в стандартной для GitHub директории:

```
.github/workflows/deploy-hw6.yml
```

GitHub Actions запускается при push в ветку hw6, проверяет TypeScript-проект, копирует приложение на сервер, запускает Docker Compose и выполняет smoke/e2e-проверку.

### Сервер

| Параметр               | Значение                                     |
|------------------------|----------------------------------------------|
| IP                     | 84.38.180.240                                |
| Домен                  | bk.lab1.zelenkov-labs.ru                     |
| Директория приложения  | /opt/zelenkov-labs/booking-microservices-hw6 |
| Публичный URL          | https://bk.lab1.zelenkov-labs.ru             |
| Локальный порт gateway | 127.0.0.1:3106                               |

Для ДЗ6 используется отдельная директория на сервере, чтобы не смешивать этот деплой с ЛР4.

### Пользователь для деплоя

На сервере создан отдельный пользователь github-hw6. Он владеет только директорией приложения:

```
/opt/zelenkov-labs/booking-microservices-hw6
```

Для запуска Docker Compose этому пользователю не выдавался полный доступ к группе docker. Вместо этого разрешена только одна команда через sudo:

```
sudo /usr/local/sbin/deploy-hw6-compose
```

Wrapper выполняет:

```
cd /opt/zelenkov-labs/booking-microservices-hw6
docker-compose -f docker-compose.vps.yml config
docker-compose -f docker-compose.vps.yml build
docker-compose -f docker-compose.vps.yml up -d --remove-orphans api-gateway
docker-compose -f docker-compose.vps.yml run --rm seed
curl -f http://127.0.0.1:3106/health
```

Такой подход ограничивает права GitHub Actions на сервере.

## GitHub Secrets

| Secret      | Назначение                                        |
|-------------|---------------------------------------------------|
| VPS_HOST    | IP сервера: 84.38.180.240                         |
| VPS_PORT    | SSH-порт: 22                                      |
| VPS_USER    | Пользователь деплоя: github-hw6                   |
| VPS_SSH_KEY | Приватный SSH-ключ для подключения GitHub Actions |

Секреты не хранятся в репозитории и не выводятся в логах workflow.

## Docker Compose

Для VPS используется файл:

```
homeworks/hw6/app/docker-compose.vps.yml
```

| Контейнер               | Назначение                            |
|-------------------------|---------------------------------------|
| hw6-api-gateway         | публичный HTTP gateway                |
| hw6-identity-service    | пользователи и авторизация            |
| hw6-catalog-service     | рестораны, столики, пересчёт рейтинга |
| hw6-menu-service        | меню и блюда                          |
| hw6-reservation-service | бронирования                          |
| hw6-review-service      | отзывы                                |
| hw6-booking-postgres    | PostgreSQL                            |
| hw6-booking-rabbitmq    | RabbitMQ                              |
| hw6-seed                | загрузка начальных данных             |

Имена контейнеров и volumes имеют префикс hw6, чтобы не конфликтовать с контейнерами LP4. Gateway внутри контейнера слушает порт 3105, а на хосте опубликован на 127.0.0.1:3106.

## Workflow

Workflow Deploy HW6 выполняет следующие шаги:

1. Получает код репозитория.
2. Устанавливает Node.js.
3. Выполняет установку зависимостей командой `npm ci --no-audit --no-fund`.
4. Выполняет TypeScript-проверку командой `npm run typecheck`.
5. Настраивает SSH-доступ к VPS.
6. Создаёт директорию приложения на сервере.
7. Копирует `homeworks/hw6/app` на сервер через `rsync`.
8. Запускает серверный wrapper `sudo /usr/local/sbin/deploy-hw6-compose`.
9. Проверяет публичный health endpoint.
10. Запускает Postman/Newman e2e-сценарий.

Публичный health endpoint проверяется командой:

```
curl -f https://bk.lab1.zelenkov-labs.ru/health
```

## Проверка через Newman

Для проверки деплоя используется Postman-коллекция из ДЗЗ, адаптированная под микросервисную версию приложения:

```
homeworks/hw6/app/postman/booking-api.postman_collection.json  
homeworks/hw6/app/postman/booking-api.postman_environment.json
```

В workflow коллекция запускается командой:

```
npx --yes newman run "$APP_PATH/postman/booking-api.postman_collection.json" \  
-e "$APP_PATH/postman/booking-api.postman_environment.json" \  
--env-var rootUrl=https://bk.lab1.zelenkov-labs.ru \  
--env-var baseUrl=https://bk.lab1.zelenkov-labs.ru/api/v1
```

Сценарий проверяет:

- health endpoint;
- регистрацию пользователя;
- авторизацию пользователя;
- получение профиля;
- получение ресторанов;
- фильтрацию ресторанов;
- получение столиков;
- получение меню и блюд;
- создание бронирования;
- авторизацию администратора;
- перевод бронирования Pending -> Confirmed -> Completed;
- создание отзыва;
- модерацию отзыва;
- публичное получение отзывов.

Локальный проверочный прогон Newman против публичного URL завершился успешно:

```
18 requests  
36 assertions  
0 failed
```

## Дополнительная настройка сервера

На VPS всего 1 GB RAM, поэтому во время Docker build возникал риск зависания на npm ci. Для стабилизации сборки был добавлен swap:

```
/swapfile 4G
```

Проверка после настройки:

```
Swap: 4.0Gi
```

Swap прописан в /etc/fstab, поэтому сохранится после перезагрузки.

## Результат

Последний запуск GitHub Actions для ветки hw6 завершился успешно:

```
Deploy HW6 - success  
Duration: 2m08s
```

После деплоя все контейнеры hw6-\* находятся в состоянии healthy, gateway доступен локально на 127.0.0.1:3106, а публичный URL возвращает:

```
{"status":"ok"}
```

Postman/Newman-сценарий подтверждает, что после автодепоя работает не только health endpoint, но и основной бизнес-процесс приложения через gateway.